БДЗ № 1: «Классы»

Общие требования

1. Проект на git'е.
2. Наличие интерактивного диалогового интерфейса для проверки корректности разработанной программы.
3. Корректное завершение программы, как в случае штатного выхода, так и в случае невосстановимых ошибок (без утечек и без использования функций мгновенного завершения программы exit, abort, std::terminate и пр.).
4. Использование высокоуровневых подходов (например std::copy вместо memcpy, std::string вместо `char*`, исключений вместо кодов возврата и т.д.).
5. Логичная и удобная структура проекта, где каждая единица (файл/библиотека) обладает своей единой зоной ответственности (каждый класс в своих файлах .h и .cpp, диалоговые функции и main в своих).
6. Наличие средств автосборки проекта (CMake, qmake и прочие, работающие "поверх" Makefile).
7. Статический анализ кода, встроенный инструментарий в IDE (пр. VS2019: Analyze->Run Code Analysis, см. также Project -> Properties -> Configuration Properties -> Code Analysis -> Microsoft -> Active Rules) или внешние инструменты (Sonarqube + extensions, Clang Static Analyzer и д.р.) (обязательно знакомство с инструментом, исправление всех замечаний или обоснование в комментарии почему конкретное замечание исправить нельзя).
8. Динамический анализ на утечки памяти, встроенный инструментарий в IDE / библиотеки (Пр., VS2019) или внешние инструменты (valgrind, Deleaker и т.п.). Отсутствие утечек памяти и прочих замечаний анализатора.
9. Не "кривой", не избыточный, поддерживаемый и расширяемый код (разумная декомпозиция, DRY, корректное использование заголовочных файлов и т.п.).
10. Стандарт языка C++20 или C++23.
11. Сдача работ проводится только в интегрированной среде разработки (IDE) или редакторе с настроенным LSP.

Требования задачи

1. Разработка модульных тестов для классов, полное (не менее 80%) покрытие методов классов модульными тестами.
2. Обязательно использование фреймворков для тестирования решения (gtest, catch2 и пр.).
3. Структура решения:
   - проект со статически линкуемой библиотекой с классом,
   - проект консольного приложения для диалоговой отладки,
   - проект для модульного тестирования.
4. Документирование всех публичных методов класса с использованием doxygen. Документация метода должна включать в себя как минимум:
   - описание всех аргументов метода,
   - описание возвращаемого значения,
   - описание всех исключений, которые могут быть выброшены в этом методе (для каждого указать тип исключения и в каких случаях оно может возникнуть).
5. Требования к оформлению классов:
   - корректность состояния класса (например, нельзя выделить массив размером N, а в capacity записать M (M!=N)),
   - отсутствие избыточности (убрать лишние поля, внутренние методы убрать в private),
   - наличие необходимых конструкторов и деструктора,

- корректность сигнатуры методов (см. стандартные сигнатуры операторов),
- сохранение семантики перегружаемых операторов (например, оператор + должен возвращать новое значение, не меняя имеющееся),
- сохранение семантики работы с потоками ввода/вывода для перегружаемых операторов ввода/вывода.

Порядок выполнения работы

Простой класс. Выполнить разработку простого класса и его методов, а также диалоговых функций для проверки класса.

Простой класс должен обладать как минимум следующими методами:
- пустой конструктор (явный или неявный);
- инициализирующие конструкторы, перечисленные в задании;
- методы получения значений атрибутов класса (геттеры);
- методы изменения значений атрибутов класса (сеттеры);
- методы ввода и вывода состояния класса в входной/выходной поток;
- прочие методы, перечисленные в задании.

Сложный класс. Выполнить разработку сложного класса и его методов, а также диалоговых функций для проверки класса.

Сложный класс должен обладать как минимум следующими методами:
- пустой конструктор (явный или неявный);
- инициализирующие конструкторы, перечисленные в задании;
- методы ввода и вывода состояния класса в входной/выходной поток;
- необходимые методы для соблюдения правила пяти или ноля;
- прочие методы, перечисленные в задании.

Перегрузка операторов. Модифицировать классы, перегрузив для них следующие операторы (внутри операторов целесообразно вызывать методы, реализованные в предыдущих пунктах):
- “>>” для ввода экземпляра простого класса из входного потока;
- “<<” для вывода экземпляра простого класса в выходной поток;
- “>>” для ввода экземпляра сложного класса из входного потока;
- “<<” для вывода экземпляра сложного класса в выходной поток;
- прочие операторы, определённые в задании (в скобках перед описанием метода).

Тестирование. В процессе выполнения каждого пункта задания необходимо реализовать тесты для разрабатываемых классов. Тесты должны покрывать все методы разработанных в некотором пункте классов. При переходе между пунктами тесты необходимо обновлять в соответствии с внесёнными изменениями.

Документация. В процессе выполнения каждого пункта вести документацию разработанных классов (см. требования), особенно перед передачей кода другому студенту.

Порядок сдачи

1. Выполнить лабораторную работу;
2. Загрузить работу на git.dozen.mephi.ru по всем правилам, указанным в отдельной инструкции;
3. Выполнить ревью 3-х работ от других студентов при помощи git.dozen.mephi.ru;
4. Внести правки по замечаниям, полученных от ваших ревьюверов, или обосновать почему эти замечания не состоятельны;
5. Выполнить сдачу решения преподавателю.

Вариант № 12: «Временная Диаграмма»

Простой класс: Троичный сигнал – может находиться в одном из трёх состояний: 0, 1 и X (неопределённое значение) и обладает длительностью в (условных единицах).

Методы простого класса (помимо общих):

- создание экземпляра класса с инициализацией значением и длительностью;
- создание экземпляра класса с инициализацией строкой (например, строка “1111”, должна привести к созданию сигнала в состоянии 1 длительностью в 4 у.е.; если строка состоит из различных символов, необходимо учитывать только первую группу одинаковых символов);
- разделение сигнала на два в указанный момент времени;
- увеличение продолжительности сигнала на заданное значение;
- уменьшение продолжительности сигнала на заданное значение;
- форматирование сигнала в виде графика (например, для 1 использовать символ “ ‾”, для 0 – “_”, а для X –“-”, продублированные нужное число раз).

Сложный класс: Временная диаграмма – определяется последовательностью троичных сигналов.

Методы сложного класса (помимо общих):

- создание экземпляров класса с инициализацией временной диаграммы постоянным уровнем сигнала и его длительностью;
- создание экземпляров класса с инициализацией временной диаграммы сигналом, представленным строкой ASCII-символов;
- (+=) объединение временной диаграммы с другой временной диаграммой;
- замена заданного промежутка времени диаграммы другой диаграммой;
- (*) дублирование временной диаграммы определённое число раз;
- (>>) сдвиг временной диаграммы на определённое время вправо;
- (<<) сдвиг временной диаграммы на определённое время влево.